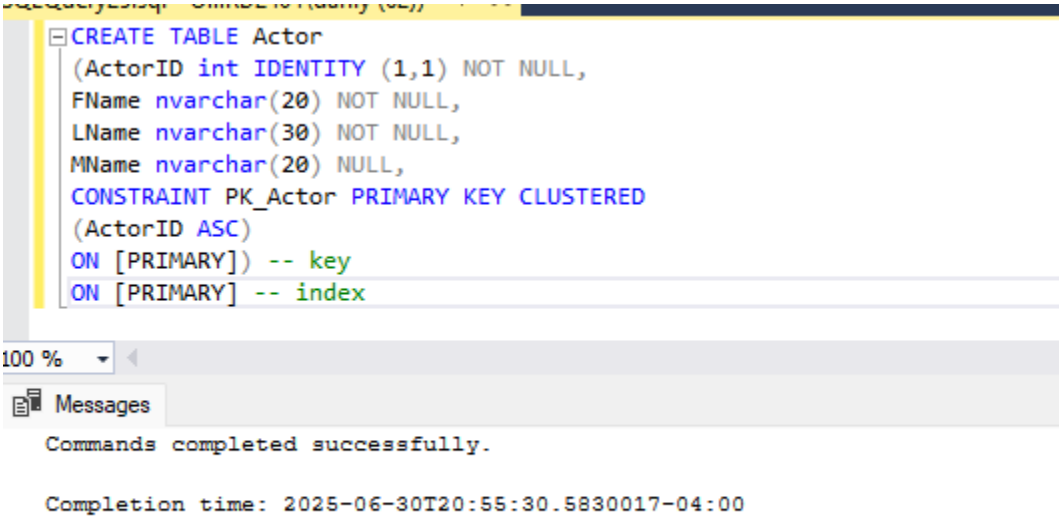
1) SQL to create an Alumni table using appropriate data types that has the following columns: AlumID, First Name, Last Name, Date of Birth, Graduation Date, Gender, Net Worth. Alum ID must be defined as an identity field that starts with 100.



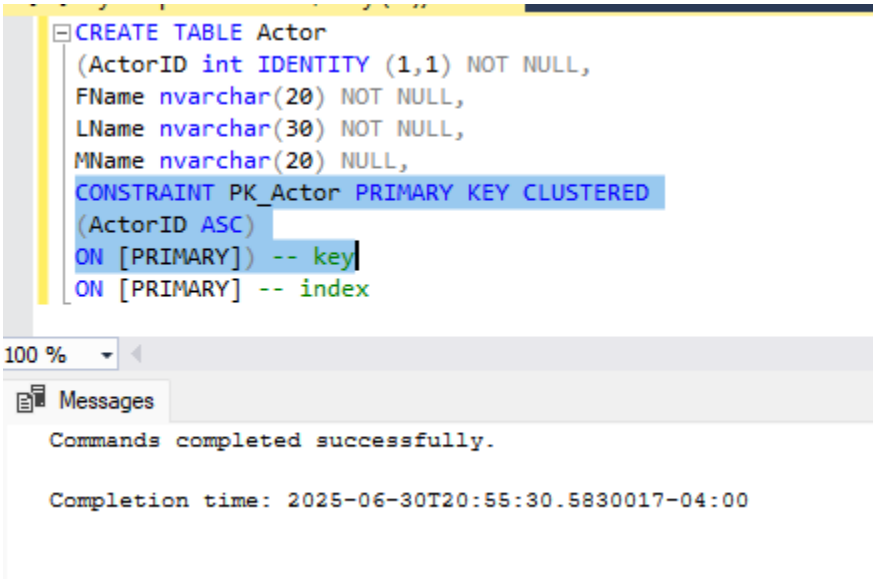
```
CREATE TABLE Actor
(ActorID int IDENTITY (1,1) NOT NULL,
FName nvarchar(20) NOT NULL,
LName nvarchar(30) NOT NULL,
MName nvarchar(20) NULL,
CONSTRAINT PK_Actor PRIMARY KEY CLUSTERED
(ActorID ASC)
ON [PRIMARY]) -- key
ON [PRIMARY] -- index
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2025-06-30T20:55:30.5830017-04:00
```

2) SQL to define the primary key in the AlumID column. (SQL must not be generated)



```
CREATE TABLE Actor
(ActorID int IDENTITY (1,1) NOT NULL,
FName nvarchar(20) NOT NULL,
LName nvarchar(30) NOT NULL,
MName nvarchar(20) NULL,
CONSTRAINT PK_Actor PRIMARY KEY CLUSTERED
(ActorID ASC)
ON [PRIMARY]) -- key
ON [PRIMARY] -- index
```

100 %

Messages

```
Commands completed successfully.

Completion time: 2025-06-30T20:55:30.5830017-04:00
```

3) SQL to define an index using the primary key and a second index using Last Name and First Name. (SQL must not be generated)

```
create index IX_Actor_LName_FName
On actor (LName, FName);
```

```
Messages
Commands completed successfully.

Completion time: 2025-06-30T21:01:38.5404811-04:00
```

4) SQL to insert ten rows of data into the Alumni table.

```
Insert into actor
    (LName, FName)
Values
    ('Daniyal','Anwar'),
    ('Dylan','Greenberg')
```

```
Messages

(2 rows affected)

Completion time: 2025-06-30T21:05:09.6015313-04:00
```

5) SQL to truncate and drop the Alumni table.

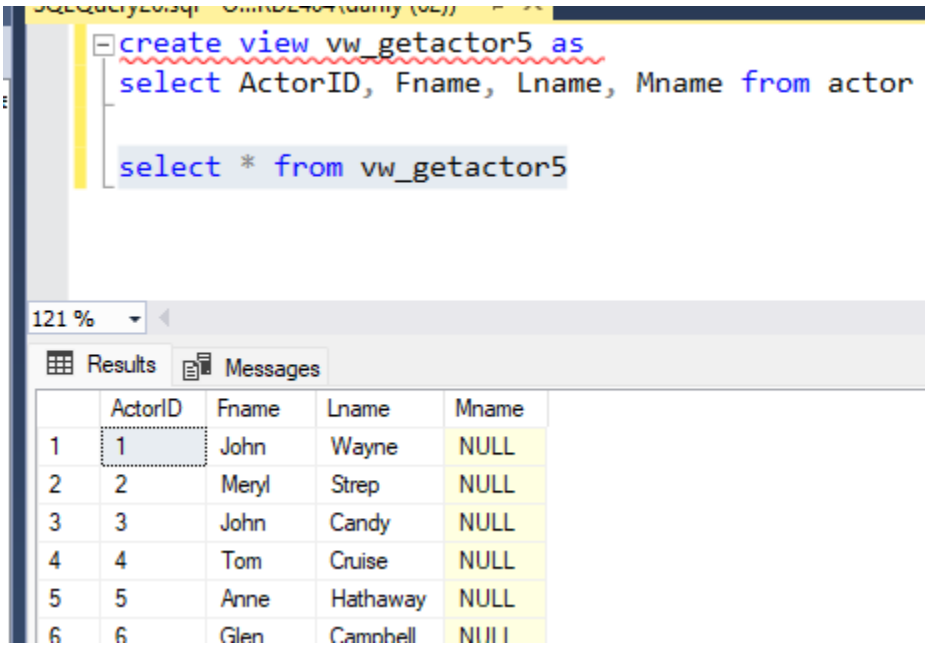
```
truncate table actor
```

```
Messages
Commands completed successfully.

Completion time: 2025-06-30T21:06:33.5427677-04:00
```

```
drop table actor
```

```
Messages
Commands completed successfully.

Completion time: 2025-06-30T21:07:23.8070462-04:00
```

6) SQL to create a view of the Alumni table using all of the columns in the table except age.



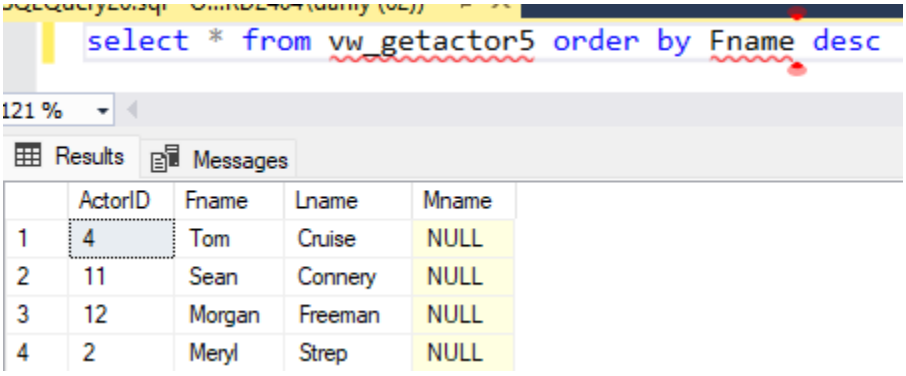
```
create view vw_getactor5 as
select ActorID, Fname, Lname, Mname from actor

select * from vw_getactor5
```

121 %

Results | Messages

	ActorID	Fname	Lname	Mname
1	1	John	Wayne	NULL
2	2	Meryl	Strep	NULL
3	3	John	Candy	NULL
4	4	Tom	Cruise	NULL
5	5	Anne	Hathaway	NULL
6	6	Glen	Campbell	NULL

7) SQL that uses the view to display all customers in the Alumni table sorted by First Name in descending order.



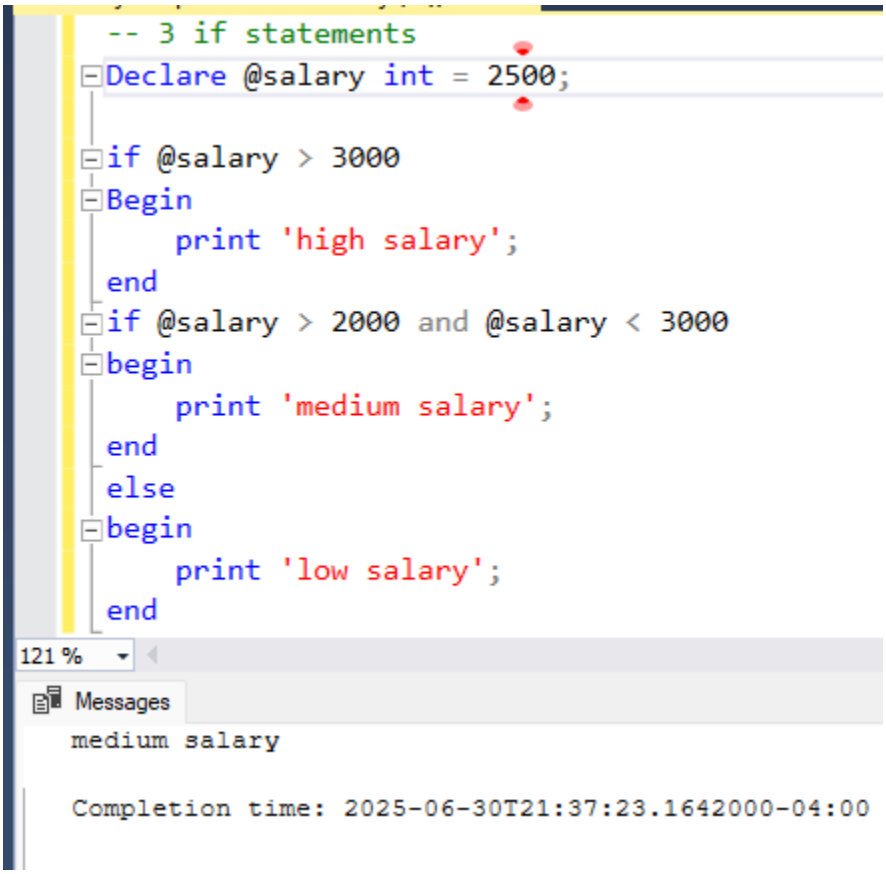
```
select * from vw_getactor5 order by Fname desc
```

121 %

Results | Messages

	ActorID	Fname	Lname	Mname
1	4	Tom	Cruise	NULL
2	11	Sean	Connery	NULL
3	12	Morgan	Freeman	NULL
4	2	Meryl	Strep	NULL

8) One sql routine with at least 3 If statements and confirmation that they work.



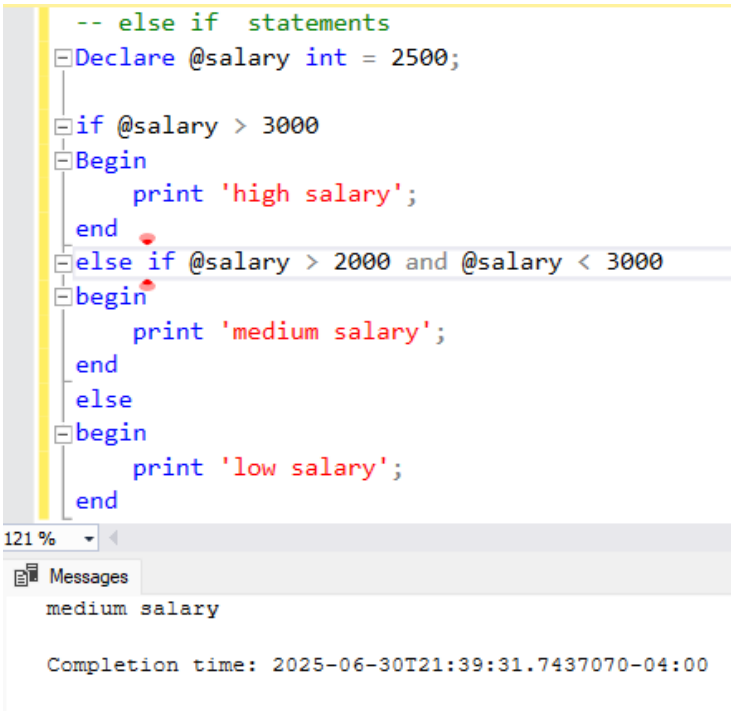
```
-- 3 if statements
Declare @salary int = 2500;

if @salary > 3000
Begin
    print 'high salary';
end
if @salary > 2000 and @salary < 3000
begin
    print 'medium salary';
end
else
begin
    print 'low salary';
end
```

121 %

Messages

```
medium salary

Completion time: 2025-06-30T21:37:23.1642000-04:00
```

9) One sql routine with If, Else-If and confirmation that they work.



```
-- else if  statements
Declare @salary int = 2500;

if @salary > 3000
Begin
    print 'high salary';
end
else if @salary > 2000 and @salary < 3000
begin
    print 'medium salary';
end
else
begin
    print 'low salary';
end
```

121 %

Messages

```
medium salary

Completion time: 2025-06-30T21:39:31.7437070-04:00
```

10) One Case statement with at least 3 When options and confirmation that they work.

```
-- case statement with 3 when options
select
    fname,
    lname,
    age,
    case
        when age > 80 then 'veteran actor'
        when age > 50 and age < 80 then 'experienced actor'
        when age < 50 and age > 40 then 'New actor'
        else 'unknown'
end as agecategory
from Actor;
```

121 %

Results | Messages

	fname	lname	age	agecategory
1	John	Wayne	118	veteran actor
2	Meryl	Strep	75	experienced actor
3	John	Candy	76	experienced actor
4	Tom	Cruise	63	experienced actor

11) Alter table sql to add an integer column to an existing table that allows nulls.

```
-- altering the table
alter table actor
add numchildren int null;
```

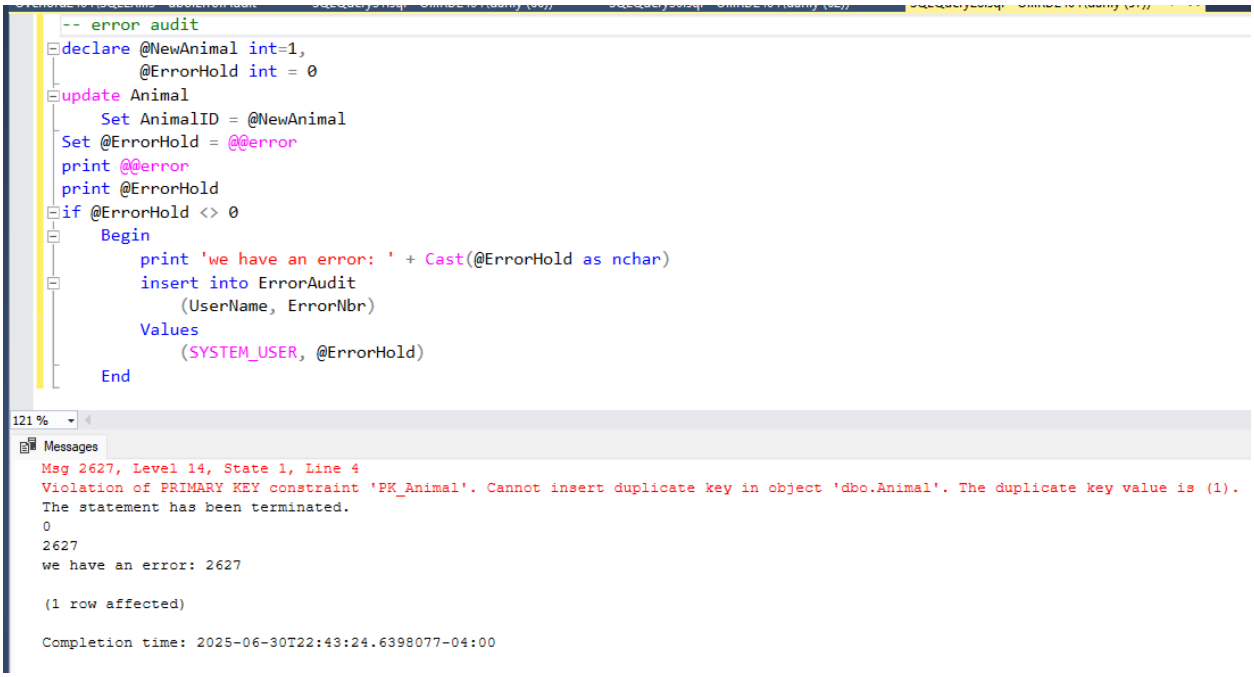
121 %

Messages

```
Commands completed successfully.

Completion time: 2025-06-30T22:08:39.7072280-04:00
```

Extra Credit from 12 to 17

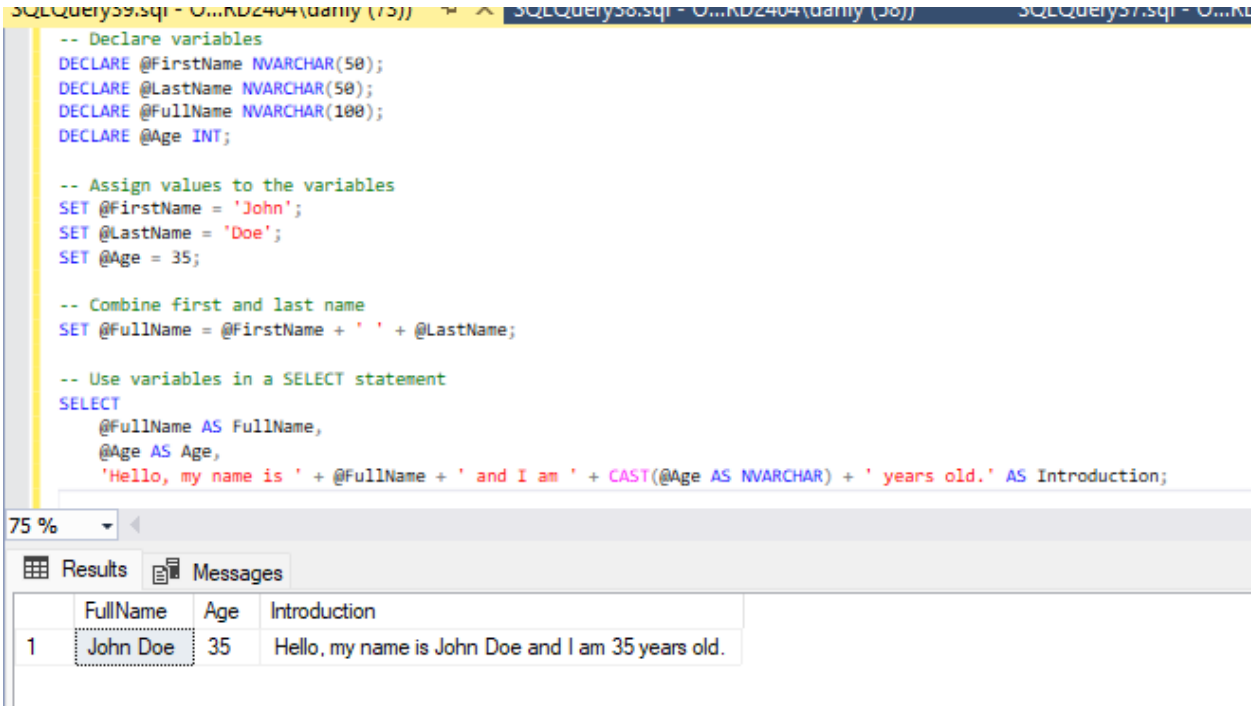
Extra Credit 12) SQL update routine with error checking and error logging. Confirmation statement that generates logged error.



```
-- error audit
declare @NewAnimal int=1,
        @ErrorHold int = 0
update Animal
    Set AnimalID = @NewAnimal
Set @ErrorHold = @@error
print @@error
print @ErrorHold
if @ErrorHold <> 0
    Begin
        print 'we have an error: ' + Cast(@ErrorHold as nchar)
        insert into ErrorAudit
            (UserName, ErrorNbr)
        Values
            (SYSTEM_USER, @ErrorHold)
    End
```

```
121 %
Messages
Msg 2627, Level 14, State 1, Line 4
Violation of PRIMARY KEY constraint 'PK_Animal'. Cannot insert duplicate key in object 'dbo.Animal'. The duplicate key value is (1).
The statement has been terminated.
0
2627
we have an error: 2627

(1 row affected)

Completion time: 2025-06-30T22:43:24.6398077-04:00
```

Extra credit 13) SQL statement that defines and uses variables.



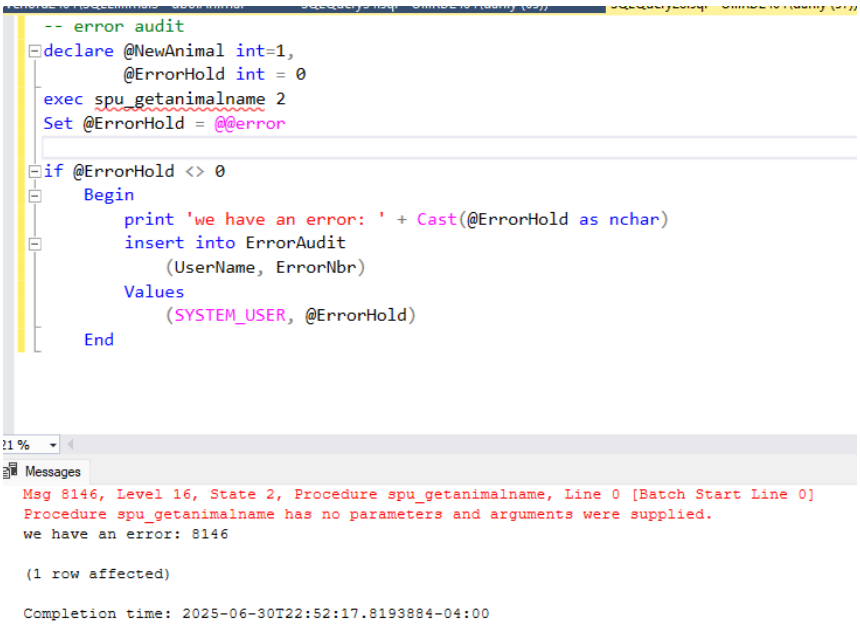
```
-- Declare variables
DECLARE @FirstName NVARCHAR(50);
DECLARE @LastName NVARCHAR(50);
DECLARE @FullName NVARCHAR(100);
DECLARE @Age INT;

-- Assign values to the variables
SET @FirstName = 'John';
SET @LastName = 'Doe';
SET @Age = 35;

-- Combine first and last name
SET @FullName = @FirstName + ' ' + @LastName;

-- Use variables in a SELECT statement
SELECT
    @FullName AS FullName,
    @Age AS Age,
    'Hello, my name is ' + @FullName + ' and I am ' + CAST(@Age AS NVARCHAR) + ' years old.' AS Introduction;
```

75 %

Results | Messages

	FullName	Age	Introduction
1	John Doe	35	Hello, my name is John Doe and I am 35 years old.

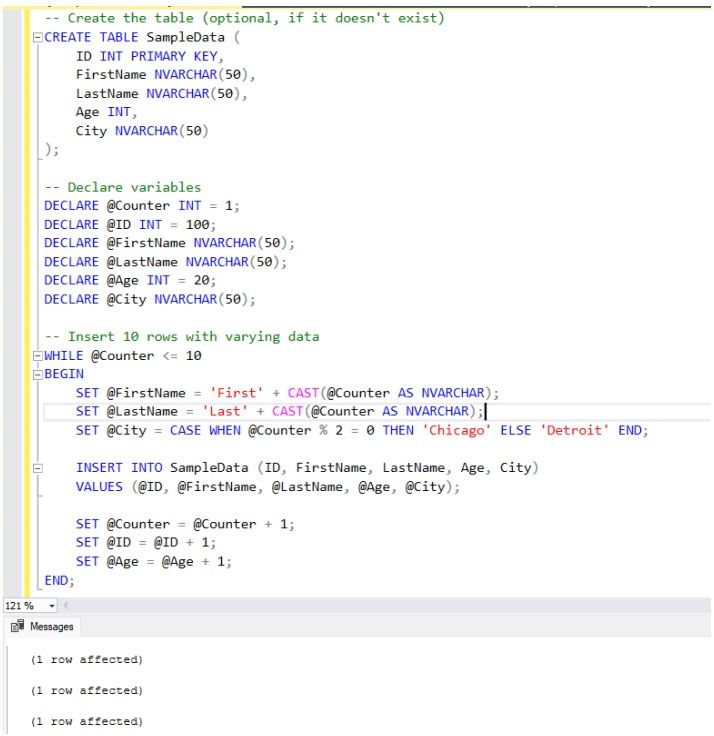
extra credit 14) Store procedure that accepts a variable and has error checking and logging as part of the execute stored procedure statement.



```
-- error audit
declare @NewAnimal int=1,
        @ErrorHold int = 0
exec spu_getanimalname 2
Set @ErrorHold = @@error

if @ErrorHold <> 0
    Begin
        print 'we have an error: ' + Cast(@ErrorHold as nchar)
        insert into ErrorAudit
            (UserName, ErrorNbr)
        Values
            (SYSTEM_USER, @ErrorHold)
    End
```

```
Messages
Msg 8146, Level 16, State 2, Procedure spu_getanimalname, Line 0 [Batch Start Line 0]
Procedure spu_getanimalname has no parameters and arguments were supplied.
we have an error: 8146

(1 row affected)

Completion time: 2025-06-30T22:52:17.8193884-04:00
```

15) SQL insert statement that inserts ten rows of data that includes at least 5 columns and the data varies for each record by using variables that are incremented and/or changed within the routine for each record.



```
-- Create the table (optional, if it doesn't exist)
CREATE TABLE SampleData (
    ID INT PRIMARY KEY,
    FirstName NVARCHAR(50),
    LastName NVARCHAR(50),
    Age INT,
    City NVARCHAR(50)
);

-- Declare variables
DECLARE @Counter INT = 1;
DECLARE @ID INT = 100;
DECLARE @FirstName NVARCHAR(50);
DECLARE @LastName NVARCHAR(50);
DECLARE @Age INT = 20;
DECLARE @City NVARCHAR(50);

-- Insert 10 rows with varying data
WHILE @Counter <= 10
BEGIN
    SET @FirstName = 'First' + CAST(@Counter AS NVARCHAR);
    SET @LastName = 'Last' + CAST(@Counter AS NVARCHAR);
    SET @City = CASE WHEN @Counter % 2 = 0 THEN 'Chicago' ELSE 'Detroit' END;

    INSERT INTO SampleData (ID, FirstName, LastName, Age, City)
    VALUES (@ID, @FirstName, @LastName, @Age, @City);

    SET @Counter = @Counter + 1;
    SET @ID = @ID + 1;
    SET @Age = @Age + 1;
END;
```

```
Messages
(1 row affected)

(1 row affected)

(1 row affected)
```

16) Create a table that has the appropriate structure to remove duplicates when loading the table. Demonstrate that it works. Include sql that defines the structure to remove duplicates in your submission. (this is the same as any other table create except you set the ignore_dup_key option to on).

```sql
-- Drop the table if it already exists
IF OBJECT_ID('dbo.UniqueUsers', 'U') IS NOT NULL
    DROP TABLE dbo.UniqueUsers;

-- Create the table with a UNIQUE constraint that ignores duplicates
CREATE TABLE dbo.UniqueUsers (
    UserID INT NOT NULL,
    FirstName NVARCHAR(50),
    LastName NVARCHAR(50),
    Email NVARCHAR(100),
    Age INT,
    CONSTRAINT UQ_UserID UNIQUE(UserID) WITH (IGNORE_DUP_KEY = ON)
);

-- Insert distinct records
INSERT INTO dbo.UniqueUsers (UserID, FirstName, LastName, Email, Age)
VALUES
(1, 'Alice', 'Smith', 'alice@example.com', 30),
(2, 'Bob', 'Johnson', 'bob@example.com', 25),
(3, 'Charlie', 'Brown', 'charlie@example.com', 28);

-- Attempt to insert a duplicate UserID (will be ignored, no error raised)
INSERT INTO dbo.UniqueUsers (UserID, FirstName, LastName, Email, Age)
VALUES
(2, 'Duplicate', 'User', 'duplicate@example.com', 99), -- Duplicate UserID
(4, 'David', 'White', 'david@example.com', 35);         -- New UserID

-- Check the final result
SELECT * FROM dbo.UniqueUsers;
```

21 %

Results | Messages

	UserID	FirstName	LastName	Email	Age
1	1	Alice	Smith	alice@example.com	30
2	2	Bob	Johnson	bob@example.com	25
3	3	Charlie	Brown	charlie@example.com	28
4	4	David	White	david@example.com	35

17) Define a table that has 10 records and 3 of the records are duplicates. Create an SQL routine that identifies the duplicate records by key value. Create a second SQL routine that displays only the duplicate records.

```sql
-- Drop table if it exists
IF OBJECT_ID('dbo.Employee', 'U') IS NOT NULL
    DROP TABLE dbo.Employee;

-- Create the Employee table
CREATE TABLE dbo.Employee (
    EmpID INT,
    FirstName NVARCHAR(50),
    LastName NVARCHAR(50)
);

-- Insert 10 rows (including 3 duplicate EmpIDs)
INSERT INTO dbo.Employee (EmpID, FirstName, LastName)
VALUES
(1, 'Alice', 'Smith'),
(2, 'Bob', 'Johnson'),
(3, 'Charlie', 'Brown'),
(4, 'David', 'White'),
(5, 'Eva', 'Taylor'),
(3, 'Chuck', 'Brown'),      -- Duplicate EmpID 3
(2, 'Bobby', 'Jones'),      -- Duplicate EmpID 2
(4, 'Dave', 'Wright'),      -- Duplicate EmpID 4
(6, 'Frank', 'Green'),
(7, 'Grace', 'Black');

-- Identify duplicate key values (EmpID)
SELECT EmpID, COUNT(*) AS Occurrences
FROM dbo.Employee
GROUP BY EmpID
HAVING COUNT(*) > 1;

-- Display only the duplicate records
SELECT *
FROM dbo.Employee
WHERE EmpID IN (
    SELECT EmpID
    FROM dbo.Employee
    GROUP BY EmpID
    HAVING COUNT(*) > 1
);
```

75 %

Results | Messages

	EmpID	Occurrences
1	2	2
2	3	2
3	4	2

	EmpID	FirstName	LastName
1	2	Bob	Johnson
2	3	Charlie	Brown
3	4	David	White
4	3	Chuck	Brown
5	2	Bobby	Jones
6	4	Dave	Wright